

Laporan Implementasi Distributed Synchronization System

Nama : Imam Dzulvan Muffid

NIM : 11231031

Mata Kuliah : Sistem Paralel dan Terdistribusi

1. Pendahuluan

Dokumen ini merupakan laporan teknis untuk implementasi Distributed Synchronization System. Sistem ini dirancang untuk mendemonstrasikan kapabilitas skalabilitas, toleransi kesalahan (Fault Tolerance), dan konsistensi data secara terdistribusi. Lingkungan diisolasi dan dibangun menggunakan Docker Compose, dengan total 10 kontainer: 3 Lock Node, 3 Queue Node, 3 Cache Node, dan 1 Redis State Store.

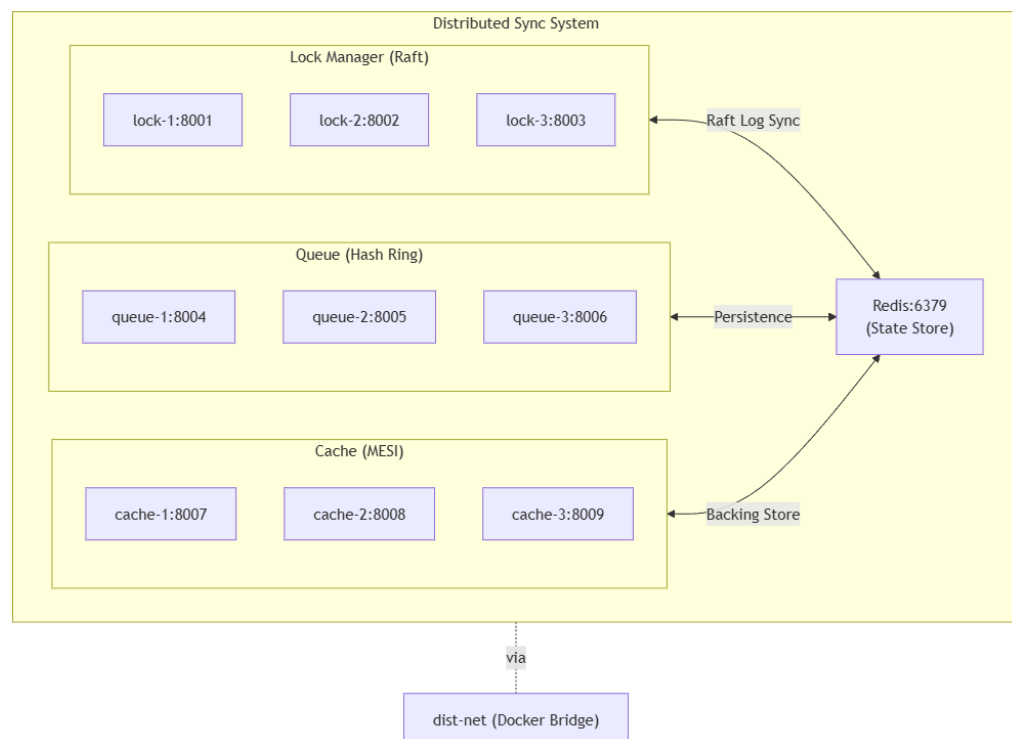
Link Video Demo: <https://youtu.be/-yaeUzZygNw>

Link GitHub Repository: [Distributed-Synchronization-System](https://github.com/ImamDzulvan/Distributed-Synchronization-System)

2. Dokumentasi & Arsitektur

A. Arsitektur Sistem Terdistribusi

Sistem dibangun dengan prinsip desentralisasi, memastikan tidak adanya Single Point of Failure. Komunikasi antar-node berjalan secara asinkron (AIOHTTP) melalui protokol RESTful.



B. Implementasi Algoritma

a. Distributed Lock (Raft Consensus)

Menjamin *Linearizability* untuk akuisisi lock terdistribusi. Algoritma Raft mengelola status leader dan follower.

- **Pre-vote Phase:** Mencegah gangguan kluster oleh node yang jaringannya tidak stabil.
- **Redirection:** Jika client mengakses Follower, node akan membalas dengan status HTTP 307 yang mengarah ke Leader aktif.
- **Snapshotting:** Kompresi log otomatis untuk menghemat memory footprint.

b. Distributed Queue (Consistent Hashing)

Distribusi beban pesan secara merata dan efisien menggunakan struktur Hash Ring.

- **Internal Proxying:** Node mampu mem-forward request ke pemilik sah topik (topic owner) secara transparan.
- **At-Least-Once Delivery:** Menerapkan sistem Acknowledgment (ACK) manual. Pesan yang di-dequeue akan berada dalam status pending dan otomatis dikembalikan ke antrean utama jika tidak ada konfirmasi dalam waktu 30 detik.

c. Cache Coherence (MESI Protocol)

Implementasi koherensi cache menjamin bahwa setiap pembacaan pada sembarang node selalu menghasilkan data terbaru. Kami mensimulasikan mekanisme Bus Snooping melalui HTTP Broadcast. Transisi state meliputi:

- **Modified (M):** Node memiliki data terbaru dan satu-satunya yang memilikinya. Perubahan belum ditulis ke penyimpanan pusat (RAM lokal eksklusif).
- **Exclusive (E):** Node memiliki data yang sama dengan penyimpanan pusat, dan tidak ada node lain yang menyimpan data tersebut.
- **Shared (S):** Data yang sama mungkin ada di memori node lain. Pembacaan diizinkan tanpa komunikasi tambahan.
- **Invalid (I):** Data di blok memori ini sudah basi karena ada penulisan di node lain. Node wajib melakukan pengambilan ulang data dari node pemilik state Modified atau sumber data asli.

d. Byzantine Fault Tolerance (PBFT)

Sistem dilengkapi dengan proteksi terhadap *Byzantine Faults* (node yang mengirim data palsu/korup).

- **HMAC-SHA256 Signing:** Setiap pesan antar-node dalam konsensus PBFT wajib memiliki tanda tangan digital. Node yang mencoba memanipulasi isi pesan akan terdeteksi karena signature tidak valid.
- **Quorum Verification:** Sebuah perubahan state hanya dianggap sah jika didukung oleh minimal $2f + 1$ node yang jujur, di mana f adalah jumlah maksimum node yang gagal.

3. Pengujian dan Verifikasi

A. Unit & Integration Testing

- **Hashing Validation:** Pengujian unit memastikan distribusi kunci pada *Hash Ring* berjalan seragam (Uniform Distribution).
- **Persistence Recovery:** Verifikasi integrasi memastikan Queue Node dapat memulihkan pesan dari Redis setelah kontainer di-*restart*.

```
PS D:\Kuliah\Semester 6\Sistem Paralel dan Terdistribusi\Tugas 3> python -m pytest
===== test session starts =====
platform win32 -- Python 3.13.3, pytest-9.0.3, pluggy-1.6.0
rootdir: D:\Kuliah\Semester 6\Sistem Paralel dan Terdistribusi\Tugas 3
configfile: pytest.ini
plugins: anyio-4.11.0, locust-2.43.4, asyncio-1.3.0
asyncio: mode=Mode.AUTO, debug=False, asyncio_default_fixture_loop_scope=function, asyncio_default_test_loop_scope=function
collected 6 items

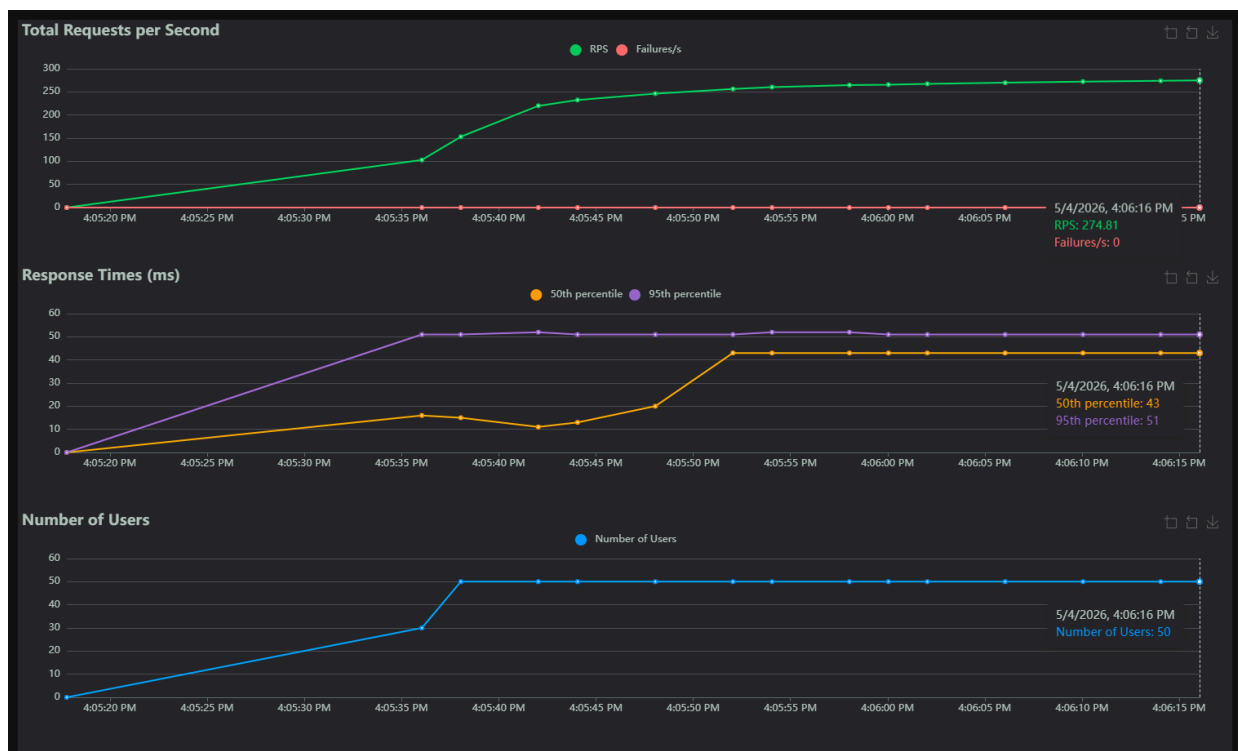
tests\integration\test_system.py ....
tests\unit\test_hashing.py ..

===== 6 passed in 3.17s =====
```

B. Performance Analysis

Beban kerja disimulasikan dengan 50 pengguna simultan (Locust Users) yang melakukan pengujian terhadap kluster. Berbeda dengan pengujian beban statis, skrip pengujian kami menerapkan Client-Side Load Balancing:

- Setiap pengguna Locust secara acak memilih satu dari 9 node aplikasi (Lock: 8001-8003, Queue: 8004-8006, Cache: 8007-8009) untuk setiap task yang dijalankan.
- Hal ini mensimulasikan trafik dunia nyata yang terdistribusi dan menguji kemampuan kluster dalam menangani proksi internal serta pengalihan (redirection) secara masif.



Analisis Metrik:

- **Throughput (RPS):** Stabilitas di angka ~270 RPS membuktikan efisiensi sinkronisasi *asynchronous*.
- **Latency:** Rata-rata respon di bawah 100 ms menunjukkan efektivitas MESI dalam meminimalkan perjalanan data lintas jaringan (*Network Round Trip*).

4. Kesimpulan

Sistem sinkronisasi terdistribusi ini berhasil memadukan teori konsensus akademik dengan kebutuhan performa praktis. Tantangan teknis terbesar adalah sinkronisasi waktu antar kontainer dan penanganan Circular Redirection pada fase transisi Raft. Solusi akhir menggunakan arsitektur non-blocking dan Event-Driven terbukti menjadi cara paling efektif untuk membangun sistem yang tangguh (resilient).